Technical report — `RedactGate × Apertus`

A deeper write-up than the README: what we built, how it works, and what the numbers say.

- **Track:** `Track 2B — RedactGate × Apertus: a sovereign privacy gateway for Swiss client data`
- **Event:** Online
- **Team:** `RedactGate` — `{{TEAM_MEMBERS}}` (INNOVA CONSULT)
- **Demo:** `{{VIDEO_URL}}`

1. Summary

Swiss fiduciaries, law firms and tax advisers handle client files full of AHV numbers, IBANs, company UIDs, birth dates and addresses. Their staff want an LLM for the routine work (summaries, task lists, letters), and the documents cannot leave the firm. RedactGate is a self-hosted, OpenAI-compatible privacy gateway: every request is scanned, each sensitive value is replaced by a reversible placeholder such as `[[AHV_6985b6]]` (AES-256-GCM vault, same value gives the same placeholder), the redacted prompt goes to the model, and the answer is re-inflated with the real values before it reaches the employee. Each request leaves a hash-chained audit event with entity counts only.

During Hack Apertus we put Apertus in both model roles of the gateway and made the whole chain run on one machine with no internet route: Apertus v1.5 8B (GGUF, llama.cpp) is the answering model **and** the second-opinion PII detector that finds names, companies and addresses in German, French and Italian text. We added a Swiss jurisdiction pack (AHV/AVS with EAN-13 check, CH/LI IBAN with mod-97, UID/IDE/IDI with mod-11, +41 phones, passport and identity-card numbers, birth dates with de/fr/it cues) and a placeholder-repair step for the small model. On the synthetic Swiss dossier in `data/`, none of the 26 sensitive values reaches the answering model, the 9 placeholders in Apertus' answer are all restored, and the audit chain detects a manipulated event. On 30 generated client notes, adding the Apertus NER pass cuts the share of sensitive values that reach the model from 42.5 % (regex packs alone) to 5.8 %; task fidelity is 95.6 % through the gateway versus 98.9 % when the raw note goes straight to the model (section 5).

2. Architecture

```
employee / any OpenAI client
   |  POST /v1/chat/completions (OpenAI schema)
   v
RedactGate (FastAPI, on-premise)
   1  normalise text (Unicode / zero-width folding)
   2  detect, concurrently:
        regex packs GENERIC + CH + EU     check-digit validated
        Apertus LLM-NER pass              raw text, stays on the machine
   3  merge spans (score, length, source priority regex > llm)
   4  vault: value -> [[TYPE_hex]]        HKDF per-session key, AES-256-GCM
   5  forward the redacted payload  ------------------------------+
   6  repair degraded tokens, re-inflate the answer (stream-safe)  |
   7  audit event: counts + HMAC hash chain, never values          |
                                                                   v
                        llama.cpp server: Apertus v1.5 8B (GGUF Q4_K_M)
                        OpenAI-compatible /v1/chat/completions
```

Both Apertus roles use the same local endpoint. The provider for each role is a setting: `AI_PROVIDER=apertus` for answers and `LLM_NER_PROVIDER=apertus` for detection. A firm can keep detection on the local Apertus and still send redacted prompts to another model; the raw text then never leaves the building, even for NER.

New code in this hackathon (branch `hack-apertus`):

File	What it does
`app/gateway/apertus.py`	Apertus adapter for any OpenAI-compatible server; reads `LLM_BASE_URL` / `LLM_NAME` / `LLM_API_KEY`; adds the placeholder instruction; repairs degraded tokens

File	What it does
`app/packs/ch.yaml`, `app/redaction/validators.py`	Swiss pack and its validators `ahv_ean13`, `ch_iban`, `ch_uid`, `ch_phone`
`app/redaction/llm_ner.py`	multilingual NER prompt with one worked example, JSON-schema constrained output, salvage of truncated replies, separate `llm_ner_provider`
`app/config.py`	`apertus` provider settings, `llm_ner_provider`
`track_2b/src/demo.py`	end-to-end demo with wire capture, leak checks and audit tamper test
`track_2b/src/eval_ch.py`	generated de/fr/it evaluation set and metrics
`track_2b/docker-compose.yml`	air-gapped deployment used by `make run`
`tests/test_ch_pack.py`, `tests/test_apertus.py`	93 new unit tests

The gateway core (proxy routes, vault, audit chain, policy engine, CA/US/EU/UA packs, multi-provider adapters) existed before the event; see section 6.

Target architecture (mandatory)

b) Air-gapped, which also covers **a) On-premise**.

`make run` starts three containers. `fetch-model` is a setup step and the only container with internet access: it downloads the GGUF once and verifies its SHA-256. `llm` (llama.cpp server) and `demo` (RedactGate plus the demo script) sit on a Docker network declared `internal: true`, which has no route to the internet. The demo probes `huggingface.co:443` and `1.1.1.1:443` at start-up and prints the result; in the compose run both are blocked.

	Build time / setup	Runtime
Container images	`python:3.12-slim`, `ghcr.io/ggml-org/llama.cpp:server-b11561`, `curlimages/curl:8.10.1`, PyPI packages from `pyproject.toml`	none
Model weights	one download from Hugging Face (5.06 GB), checksum pinned	read-only volume
Network	Docker Hub, GHCR, PyPI, Hugging Face	internal network only

For a production on-premise install the same app runs with Postgres for the vault and audit tables (`docker-compose.yml` at the repository root, `AI_PROVIDER=apertus`). We ran that mode once against the local Apertus: the dossier request returned HTTP 200 in 21.7 s with all values restored, Postgres held 26 encrypted `token_map` rows, a `pg_dump` of the database contained none of the sampled raw values (AHV, IBAN, name, street), and `verify_chain` returned true. The demo uses the DB-less wiring of the same FastAPI app (in-memory vault store and in-memory audit chain) so that judges need nothing besides Docker.

3. Use of Apertus

- **Model:** Apertus v1.5 8B, text-only, GGUF Q4_K_M: `Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`, a quantisation of `andreasmartin/apertus-v1.5-8b-text` (the text tower of `swiss-ai/Apertus-v1.5-8B`), Apache-2.0, SHA-256 `a037df8d87ff6caacee794ee85f55342f2152e0d359b7389033300c3bee5f299`.
- **How it is used:** inference in two roles: (1) answering model behind the gateway, (2) structured NER for free-text PII.
- **Where it runs:** local weights, llama.cpp `llama-server` build b11561, OpenAI-compatible API, `--jinja` so the chat template embedded in the GGUF is used. Context 8192, one slot.

Answer role. When the redacted prompt contains placeholders, the adapter prepends a system instruction: a token is the value, write it complete with brackets, plus one correct and three wrong examples. Without the examples Apertus 8B wrote `AHV c4857b` instead of `[[AHV_c4857b]]` in our dossier test; with them it copied all tokens verbatim. As a safety net the adapter rewrites degraded forms (`AHV c4857b`, `AHV_c4857b`, `[AHV_c4857b]`) back to the canonical token, but only for codes that were present in the request, so it can never create a mapping that did not exist.

NER role. System prompt in English listing the types (PERSON, ORG, ADDRESS, …), one synthetic German example, temperature 0, and `response_format: json_schema` with an enum of types and `maxItems: 48`. llama.cpp turns the schema into a grammar, so the reply is always valid JSON of the right shape. The cap matters: on the full dossier the unconstrained model once looped on the same IBAN until `max_tokens`, which made the JSON invalid and dropped every entity; the gateway now also salvages complete items from a truncated reply. Returned strings are re-anchored in the source text; anything that cannot be found is discarded. Merged with the regex spans, regex wins on overlap (higher score and source priority).

Other models. None. Development and evaluation used only Apertus; the evaluation metrics are string matches, no LLM judge.

4. Data

All data is synthetic and generated by us; there is no personal data in the repository.

- `data/swiss_dossier.json`: one fiduciary dossier with three clients (German, French, Italian section), 26 labelled sensitive values. Names use the Swiss placeholder names Muster / Exemple / Esempio, e-mails use the reserved `example.org` domain, and every identifier was computed with a valid check digit for an invented body.
- `data/eval_set.jsonl`: 30 client notes produced by `src/eval_ch.py` with seed 7 (10 per language, two templates each), every value labelled with its type.

Licence of both files: CDLA-Permissive-2.0 (event terms). Size: under 50 KB.

5. Evaluation

Setup. Apertus v1.5 8B Q4_K_M on llama.cpp b11561, NVIDIA RTX 3060 12 GB (all layers offloaded), AMD Ryzen 5 5600G. Commands: `python track_2b/src/demo.py`, `python track_2b/src/eval_ch.py 30`.

Dossier demo (`src/demo.py`, 26 labelled values, one request through the gateway):

Check	Result
structured identifiers sent to the model in clear (AHV, IBAN, UID, phone, passport, DOB, e-mail)	0 of 17
names, companies, addresses sent in clear (Apertus NER)	0 of 9
placeholders in Apertus' answer restored	9 of 9
raw values in the audit log	0
audit chain valid / detects one edited event	yes / yes
end-to-end latency, GPU	22.6 s (NER pass + answer)
end-to-end latency, CPU only (`make run` container)	377.6 s (Docker, llama.cpp CPU build, 6 cores)

Generated notes (`src/eval_ch.py`, 30 notes, 294 labelled values). Leak rate = share of labelled values that still appear in the text sent to the model. Task fidelity = share of the three requested values (name, AHV, IBAN) present verbatim in the answer the user receives.

Setup	Leak rate	Task fidelity
Baseline: raw note straight to Apertus (no gateway)	100 % by construction	98.9 %
RedactGate, regex packs only (GENERIC + CH + EU)	42.5 %	n/a
RedactGate, regex packs + Apertus NER	5.8 %	95.6 %

Details (`docs/results/eval_report.json`, per-case answers included):

- **Regex packs alone** redact every AHV, IBAN, UID, phone, e-mail and passport number (0 leaks in 144 values). What they cannot see are names (30), companies (30), addresses (60) and five birth dates written without a keyword, as in `Lea Brunner (14.03.1981)`.
- **With the Apertus NER pass** the remaining leaks are 10 address parts, 5 company names and 2 birth dates; no person name reached the model. The NER pass produced one false positive in 30

notes (the word `IVA` tagged as a tax ID).

- **Fidelity.** Apertus wrote 76 tokens in a degraded form (for example `AHV_514251` without brackets); the adapter repaired all of them and no placeholder was left unresolved in any final answer. The 3.3-point gap to the baseline comes from four notes where the model answered with the wrong field: twice it wrote the label `Client` / `Cliente` where the name token belonged, twice it left out the AHV token. The baseline missed one value in one note.
- A second full run gave the same leak rates and 93.3 % fidelity through the gateway; at temperature 0 the GPU build is still not bit-for-bit deterministic.

Unit tests. `pytest -q`: 440 passed, 6 skipped (the skipped ones need a live Postgres). 93 of them are new: every Swiss validator with valid and invalid synthetic values, every CH pattern with positive and negative cases in de/fr/it, an end-to-end vault round trip of the dossier, the Apertus adapter against a mocked server, placeholder repair, and the NER pass with a dedicated local provider.

6. Limitations

- **Pre-existing base.** The gateway core (vault, audit chain, proxy, policy, non-Swiss packs) was written in June 2026 under AGPL-3.0. The Apertus integration, Swiss pack, NER changes, placeholder repair, demo, evaluation and deployment in `track_2b/` were built during the event (branch `hack-apertus`).
- **NER is probabilistic.** Detection of names, companies and addresses depends on the 8B model; results vary slightly between runs even at temperature 0 on GPU. Structured identifiers do not depend on the model.
- **Placeholder repair covers non-streaming answers.** Streaming answers rely on the instruction alone.
- **Passport and identity-card numbers** are matched by shape (letter + 7 digits) next to a keyword; there is no standalone check digit to validate.
- **Phone numbers** without separators in the national format (`0791234567`) are not matched, to avoid redacting arbitrary 10-digit references.
- **Synthetic data only.** We did not test on real client documents, scanned PDFs or OCR output.
- **Quantised community build.** The GGUF is a third-party Q4_K_M conversion of the text tower of the swiss-ai weights. We did not measure the quality gap to the BF16 weights or to the 70B model.
- **CPU speed.** Without a GPU the 8B model generates about 5 tokens/s on a 6-core desktop CPU, so one dossier takes minutes.

7. Reproducibility

- Hardware used: AMD Ryzen 5 5600G (6 cores), 128 GB RAM, NVIDIA RTX 3060 12 GB. `make run` uses CPU only and needs about 8 GB RAM and 6 GB disk for the weights.
- `make run` (Docker): downloads the weights once, starts llama.cpp on an internal network, runs the demo. Exit code 0 means every structured identifier was redacted and the audit chain behaved as expected. Output also lands in `out/demo_report.json`.
- `make eval`: the 30-note evaluation in the same air-gapped setup (`out/eval_report.json`).
- `make run-remote` with `LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`: the same demo against a hosted Apertus endpoint, for example a sovereign Swiss cloud deployment.
- `make test`: the unit suite (no model, no network).
- Without Docker: run `llama-server -m apertus-v1.5-8b-text-q4_k_m.gguf --jinja --port 8091`, then `pip install -e .` at the repository root and `python track_2b/src/demo.py`.
- Seed 7 for the generated notes. Code measured at commit `f31459b` on branch `hack-apertus`; later commits change documentation and the video only.

8. Next steps

- Apertus 70B through a sovereign Swiss endpoint and a side-by-side quality comparison with the local 8B.

- A small LoRA on Apertus 8B for PII span extraction in Swiss German, French and Italian administrative text, trained on generated data like `eval_set.jsonl`.
- More Swiss identifiers: cantonal tax numbers, health insurance card numbers, QR-bill references.
- Placeholder repair inside the streaming path.
- Pilot with a fiduciary office in Switzerland on real (consented) documents.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Apertus team, *Apertus: Democratizing Open and Compliant LLMs*, arXiv:2509.14233 (2025).
- swiss-ai/Apertus-v1.5-8B, https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- llama.cpp, https://github.com/ggml-org/llama.cpp
- ZAS / CdC, AHV number format (756.XXXX.XXXX.XC, EAN-13 check digit).
- Federal Statistical Office, UID register: CHE-XXX.XXX.XXX with mod-11 check digit.
- ISO 13616 (IBAN), mod-97 check.